dm1_analysis_onefile.R

ACoP 2026 Hackathon, Team 1. The DM1 analysis in one file: Part 1 = data, cleaning checks, split and helpers (R/common.R); Part 2 = the analysis (dm1_analysis.R). Code unchanged; outputs identical to the two-file run. Run from the analysis folder with the dataset in data/: **Rscript dm1_analysis_onefile.R**. Green = comments. Repository: github.com/euibeom135-cell/acop2026-hackathon-team1

```
# ACoP 2026 Hackathon, Team 1: DM1 treatment-response predictors per arm
# Question: can ML identify additional predictors and improve stratification beyond
#           traditional covariate modelling?
# Run:      Rscript dm1_analysis_onefile.R     (from the folder holding this file; dataset in data/ next to it)
# Repository: https://github.com/euibeom135-cell/acop2026-hackathon-team1  (code, decision log, AI disclosure, team pipelines)
# Output:   dm1_outputs/  scoreboard_*.csv, covariates_*.csv, fig_*.png
#
# AI disclosure: drafted with Claude (Anthropic) via Claude Code, used for code writing and
# workflow design. The team reviewed and is accountable for every step.
#
# Workflow (the six kickoff steps):
#   1 Understand   per-arm correlations and class balance, training data only
#   2 Split        per arm, 75/25, stratified by responder label, fixed seed, test set locked
#   3 Preprocess   dummy coding; scaling and imputation learned on training data only
#   4 Select       covariate models (stepwise, LASSO, elastic net) vs ML (random forest, XGBoost)
#   5 Train        tuning by 5-fold cross-validation inside the training set
#   6 Evaluate     once on the test set, with bootstrap intervals
#
# Signal vs noise: a pure-noise probe column is added as a visible negative control; each method's
# pick must beat it (random forest = permutation importance, XGBoost = SHAP). LASSO stability over
# bootstrap resamples and agreement across the five methods decide the call.
#
# Two choices informed by explorations on training data only (analysis/explorations/euibeom/):
#   "Linear with cut-off": the top covariate enters as above/below a threshold chosen in training,
#   because in the Tideglusib arm the outcome steps rather than slopes with cognitive_function.
#   Responder cut-off = the training responder share, not 0.5, because arms with few responders
#   (Mexiletine, 29%) otherwise call almost nobody a responder.
#
# READING GUIDE. This file is the two analysis files joined for review, with no change to the code:
#   Part 1 = R/common.R (data, cleaning checks, the split, metrics), Part 2 = dm1_analysis.R (the analysis).
#   Steps are numbered as in the kickoff workflow. The test set is requested in exactly one place (Part 2, step 2).


################################################################################################################
# PART 1  Data, cleaning checks, the train/test split, metric helpers   (= R/common.R)
################################################################################################################

SEED <- 20261011

# Find the analysis folder from wherever the calling script is run.
ANALYSIS_DIR <- local({ d <- normalizePath(getwd())
  repeat { if (file.exists(file.path(d, "dm1_analysis.R")) || file.exists(file.path(d, "dm1_analysis_onefile.R"))) break   # works standalone too
           if (file.exists(file.path(d, "analysis", "dm1_analysis.R"))) { d <- file.path(d, "analysis"); break }
           if (dirname(d) == d) stop("Run from inside the repository."); d <- dirname(d) }
  d })

# ---- Data ----
f <- Sys.getenv("DM1_DATA")
if (!nzchar(f)) {
  f <- list.files(file.path(ANALYSIS_DIR, "data"), pattern = "\.(xlsx|csv)$", full.names = TRUE, ignore.case = TRUE)
  f <- f[grepl("dm1", basename(f), ignore.case = TRUE) & !grepl("dictionary|mock", basename(f), ignore.case = TRUE)][1]
}
if (is.na(f)) stop("Put the hackathon dataset (DM1*.xlsx or .csv) in analysis/data/")
raw <- if (grepl("\.csv$", f, ignore.case = TRUE)) read.csv(f) else as.data.frame(readxl::read_excel(f))
names(raw) <- tolower(names(raw))

num_cov <- c("ctg_repeats", "somatic_mosaicism_index", "age", "age_at_onset", "disease_duration",
             "myotonia_severity", "muscle_strength", "pr_interval", "cognitive_function",
             "ck_level", "insulin_resistance", "albumin", "bmi")
```

```r
cat_cov <- c("phenotype", "sex", "family_history", "cataracts", "smoking_status")
covs    <- c(num_cov, cat_cov)
stopifnot("Covariate names differ from the kickoff slides; edit num_cov / cat_cov" = all(covs %in% names(raw)))
lab <- grep("^responder", names(raw), value = TRUE)
arms <- data.frame(
  name    = c("Mexiletine", "Oligonucleotide", "Tideglusib"),
  outcome = c("myotonia_change_12wk", "qmt_change_26wk", "tideglusib_composite"),
  label   = c(grep("mex", lab, value = TRUE), grep("oligo", lab, value = TRUE), grep("tid", lab, value = TRUE)),
  better  = c("lower", "higher", "higher"))

# ---- Data checks and cleaning ----
# Only two things are fixed automatically, because they cannot change a real value: stray spaces
# and inconsistent capitalisation in category labels, and exact duplicate rows. Everything else
# is reported for the team to decide on, and each decision should be written here as code.
qc <- character(); note <- function(...) qc <<- c(qc, sprintf(...))
for (v in c(cat_cov, lab)) {
  x <- trimws(as.character(raw[[v]])); key <- tolower(gsub("[ _-]+", "_", x))
  canon <- tapply(x, key, function(z) names(which.max(table(z))))      # most common spelling wins
  if (any(x != canon[key], na.rm = TRUE) || any(x != as.character(raw[[v]]), na.rm = TRUE))
    note("FIXED  %s: unified label spellings (%s)", v, paste(unique(raw[[v]][!is.na(x) & as.character(raw[[v]]) != canon[key]]), collapse = ", "))
  raw[[v]] <- ifelse(is.na(x), NA, canon[key])
}
dup <- duplicated(raw[setdiff(names(raw), names(raw)[1])])
if (any(dup)) { note("FIXED  removed %d exact duplicate rows", sum(dup)); raw <- raw[!dup, ] }
for (v in c(num_cov, arms$outcome)) if (!is.numeric(raw[[v]])) {
  bad <- unique(raw[[v]][is.na(suppressWarnings(as.numeric(raw[[v]]))) & !is.na(raw[[v]])])
  note("CHECK  %s is stored as text; non-numeric entries: %s", v, paste(head(bad, 8), collapse = ", ")); raw[[v]] <- suppressWarnings(as.numeric(raw[[v]])) }
nmiss <- colSums(is.na(raw[covs])); if (any(nmiss > 0)) note("CHECK  missing values: %s", paste(sprintf("%s (%d)", names(nmiss)[nmiss > 0], nmiss[nmiss > 0]), collapse = ", "))
for (v in num_cov) { x <- raw[[v]]; q <- quantile(x, c(.25, .75), na.rm = TRUE); out <- which(x < q[1] - 3 * diff(q) | x > q[2] + 3 * diff(q))
  if (length(out)) note("CHECK  %s: %d extreme values (beyond 3 IQR), e.g. %s; range %s to %s", v, length(out), paste(head(signif(x[out], 4), 5), collapse = ", "), signif(min(x, na.rm = TRUE), 4), signif(max(x, na.rm = TRUE), 4))
  if (any(x < 0, na.rm = TRUE) && v != "cognitive_function") note("CHECK  %s: %d negative values", v, sum(x < 0, na.rm = TRUE))   # cognitive_function is a z-score
  for (edge in range(x, na.rm = TRUE)) if (mean(x == edge, na.rm = TRUE) > 0.10)
    note("CHECK  %s: %.0f%% of patients sit exactly at %s (a floor, ceiling or detection limit)", v, 100 * mean(x == edge, na.rm = TRUE), signif(edge, 4)) }
arm_col <- intersect(c("treatment_arm", "arm", "treatment"), names(raw))[1]
if (!is.na(arm_col)) for (v in covs) for (g in unique(raw[[arm_col]])) if (length(unique(raw[[v]][raw[[arm_col]] == g])) < 2)
  note("CHECK  %s has a single value (%s) for every %s patient: no information in that arm; dropped there", v, raw[[v]][raw[[arm_col]] == g][1], g)
if (any(raw$age_at_onset > raw$age, na.rm = TRUE)) note("CHECK  %d patients have onset after current age", sum(raw$age_at_onset > raw$age, na.rm = TRUE))
gap <- abs(raw$age - raw$age_at_onset - raw$disease_duration)
if (any(gap > 1, na.rm = TRUE)) note("CHECK  %d patients where age - age_at_onset differs from disease_duration by more than 1 year", sum(gap > 1, na.rm = TRUE))
nout <- rowSums(!is.na(raw[arms$outcome])); if (any(nout != 1)) note("CHECK  %d patients do not have exactly one arm outcome", sum(nout != 1))
for (a in seq_len(nrow(arms))) { has <- !is.na(raw[[arms$outcome[a]]])
  if (any(has & is.na(raw[[arms$label[a]]]))) note("CHECK  %s: %d patients with an outcome but no responder label", arms$name[a], sum(has & is.na(raw[[arms$label[a]]]))) }
if (!length(qc)) qc <- "No issues found by the automatic checks."


# Team cleaning decisions go here, one line each with the reason, e.g.
# raw <- raw[raw$age_at_onset <= raw$age, ]          # onset after current age is impossible

for (v in cat_cov) raw[[v]] <- factor(raw[[v]])

# ---- Small helpers ----
is_resp <- function(x) { x <- tolower(as.character(x)); as.integer(x %in% c("1", "true", "yes") | grepl("^resp", x)) }
auc <- function(y, p) { r <- rank(p); n1 <- sum(y == 1); n0 <- sum(y == 0); (sum(r[y == 1]) - n1 * (n1 + 1) / 2) / (n1 * n0) }
```

```r
reg_metrics <- function(y, p) c(RMSE = sqrt(mean((y - p)^2)), MAE = mean(abs(y - p)), R2 = 1 - sum((y - p)^2) / sum((y - mean(y))^2))
cls_metrics <- function(y, p, cut = 0.5) { h <- as.integer(p >= cut)
  c(AUC = auc(y, p), Recall = sum(h & y) / sum(y), Precision = if (sum(h)) sum(h & y) / sum(h) else NA, Accuracy = mean(h == y)) }
# Median / most-frequent imputation learned on training data (no effect if nothing is missing).
impute <- function(df, ref) { for (v in names(df)) if (anyNA(df[[v]]))
  df[[v]][is.na(df[[v]])] <- if (is.numeric(ref[[v]])) median(ref[[v]], na.rm = TRUE) else names(which.max(table(ref[[v]])))
  df }
# ---- The split: one per arm, made here and nowhere else ----
# 75/25, fixed seed, stratified by responder label and onset phenotype. Imputation (if any value
# were missing) is learned from the training patients. By default only the training set is returned.
load_arm <- function(name, include_test = FALSE) {
  a <- match(name, arms$name); if (is.na(a)) stop("Arm must be one of: ", paste(arms$name, collapse = ", "))
  A <- arms[a, ]
  d <- raw[!is.na(raw[[A$outcome]]), ]
  d$.resp <- is_resp(d[[A$label]])
  set.seed(SEED + a)
  test_id <- unlist(lapply(split(seq_len(nrow(d)), interaction(d$.resp, d$phenotype, drop = TRUE)),
                           function(i) sample(i, round(0.25 * length(i)))))
  tr <- d[-test_id, ]; te <- d[test_id, ]
  tr[covs] <- impute(tr[covs], tr[covs]); te[covs] <- impute(te[covs], tr[covs])
  # Covariates that do not vary among this arm's training patients cannot predict anything there.
  varies <- covs[sapply(tr[covs], function(x) length(unique(x)) > 1)]
  out <- list(info = A, train = tr, covs = varies, dropped = setdiff(covs, varies))
  if (include_test) out$test <- te
  out
}

# Cross-validation folds inside the training set, for comparing ideas without touching the test set.
cv_folds <- function(train, k = 5) { set.seed(SEED)
  f <- integer(nrow(train)); for (g in split(seq_len(nrow(train)), train$.resp)) f[g] <- sample(rep(seq_len(k), length.out = length(g))); f }

################################################################################################
# PART 2  The analysis: setup   (= dm1_analysis.R)
################################################################################################

pkgs <- c("readxl", "dplyr", "tidyr", "ggplot2", "glmnet", "MASS", "ranger", "xgboost", "shapviz", "rpart", "rpart.plot")
miss <- setdiff(pkgs, rownames(installed.packages()))
if (length(miss)) install.packages(miss, repos = "https://cloud.r-project.org")
suppressPackageStartupMessages({ library(dplyr); library(tidyr); library(ggplot2); library(glmnet); library(ranger); library(xgboost) })
theme_set(theme_bw(base_size = 12))
OUT <- Sys.getenv("DM1_OUT", "dm1_outputs")
dir.create(OUT, showWarnings = FALSE)

# Data loading, cleaning, the train/test split and small helpers live in R/common.R, shared with explorations.

# The two files are joined above, so the data, split and helpers are already defined here.
cat("Data:", f, "-", nrow(raw), "rows,", ncol(raw), "columns
")
writeLines(qc, file.path(OUT, "data_checks.txt")); cat("
Data checks:
", paste0("  ", qc, collapse = "
"), "
", sep = "")

# ---- Helpers ----
boot_ci <- function(y, p, fun, B = 1000) { set.seed(SEED)
  b <- replicate(B, { i <- sample(length(y), replace = TRUE); tryCatch(fun(y[i], p[i]), error = function(e) NA) })
  sprintf("%.2f to %.2f", quantile(b, 0.025, na.rm = TRUE), quantile(b, 0.975, na.rm = TRUE)) }
design <- function(df, vars) { X <- model.matrix(reformulate(vars), df); a <- attr(X, "assign")[-1]; X <- X[, -1, drop = FALSE]; attr(X, "var") <- vars[a]; X }
by_var <- function(score, X) tapply(score, attr(X, "var"), function(s) sum(abs(s)))   # dummy columns back to covariates

# All models for one outcome type: test-set predictions per method, and what each method picked.
```

```r
fit_all <- function(tr, te, vars, y, family) {
  bin <- family == "binomial"
  Xtr <- design(tr, vars); Xte <- design(te, vars); ytr <- tr[[y]]
  dtr <- cbind(tr[vars], .y = if (bin) factor(ytr) else ytr); dte <- te[vars]
  pred <- list(); sel <- list()

  # Covariate models
  full <- glm(.y ~ ., data = dtr, family = if (bin) binomial else gaussian)
  step <- MASS::stepAIC(update(full, . ~ 1), scope = list(lower = ~1, upper = formula(full)),
                        direction = "both", k = log(nrow(dtr)), trace = 0)
  pred[["Stepwise (BIC)"]] <- predict(step, dte, type = "response")
  sel$Stepwise <- setNames(as.numeric(vars %in% attr(terms(step), "term.labels")), vars)
  for (m in list(c("LASSO", 1), c("Elastic net", 0.5))) {
    set.seed(SEED); cv <- cv.glmnet(Xtr, ytr, alpha = as.numeric(m[2]), family = family, nfolds = 5)
    pred[[m[1]]] <- as.numeric(predict(cv, Xte, s = "lambda.1se", type = "response"))
    sel[[m[1]]]  <- by_var(as.numeric(coef(cv, s = "lambda.1se"))[-1], Xtr)[vars]
  }

  # Machine learning
  rf <- ranger(.y ~ ., data = dtr, num.trees = 500, importance = "permutation", probability = bin, seed = SEED)
  pr <- predict(rf, dte)$predictions
  pred[["Random forest"]] <- if (bin) pr[, "1"] else pr
  sel[["Random forest"]]  <- rf$variable.importance[vars]
  d1 <- xgb.DMatrix(Xtr, label = ytr); d2 <- xgb.DMatrix(Xte)
  obj <- if (bin) "binary:logistic" else "reg:squarederror"
  set.seed(SEED); x0 <- xgb.train(list(objective = obj), d1, nrounds = 100, verbose = 0)
  pred[["XGBoost, untuned"]] <- predict(x0, d2)
  grid <- expand.grid(max_depth = c(1, 2, 3), eta = c(0.03, 0.1))      # coarse on purpose: small arms
  cvs <- lapply(seq_len(nrow(grid)), function(i) { set.seed(SEED)
    lg <- as.data.frame(xgb.cv(list(objective = obj, max_depth = grid$max_depth[i], eta = grid$eta[i], subsampl
e = 0.8),
                               d1, nrounds = 300, nfold = 5, verbose = 0)$evaluation_log)
    e <- lg[[grep("^test_.*_mean$", names(lg))[1]]]; c(best = min(e), n = which.min(e)) })
  k <- which.min(sapply(cvs, `[`, "best"))
  set.seed(SEED); x1 <- xgb.train(list(objective = obj, max_depth = grid$max_depth[k], eta = grid$eta[k], subsa
mple = 0.8),
                                  d1, nrounds = cvs[[k]][["n"]], verbose = 0)
  pred[["XGBoost, tuned"]] <- predict(x1, d2)
  shap <- predict(x1, d1, predcontrib = TRUE)
  sel[["XGBoost SHAP"]] <- by_var(colMeans(abs(shap[, colnames(Xtr), drop = FALSE])), Xtr)[vars]   # mean |SHAP
|, bias column dropped
  list(xgb = x1, Xtr = Xtr, pred = as.data.frame(pred, check.names = FALSE), sel = as.data.frame(sel, check.nam
es = FALSE, row.names = vars),
       tuned = sprintf("max_depth %d, eta %.2f, %d rounds", grid$max_depth[k], grid$eta[k], cvs[[k]][["n"]]))
}

# ---- Per-arm analysis ----
type_of <- c("Stepwise (BIC)" = "Covariate", "LASSO" = "Covariate", "Elastic net" = "Covariate", "Consensus lin
ear" = "Covariate",
             "Linear with cut-off" = "Covariate", "Random forest" = "ML", "XGBoost, untuned" = "ML", "XGBoost,
tuned" = "ML")
all_cov <- list(); all_board <- list()

for (a in seq_len(nrow(arms))) {
  A <- arms[a, ]; cat("
==========", A$name, "==========
")
  # The split is made in R/common.R (75/25, stratified, fixed seed). This is the only place the
  # locked test set is requested.
  S <- load_arm(A$name, include_test = TRUE); tr <- S$train; te <- S$test; d <- rbind(tr, te)
  covs_a <- S$covs; num_a <- intersect(num_cov, covs_a)        # this arm's usable covariates
  if (length(S$dropped)) cat("Dropped in this arm (no variation):", paste(S$dropped, collapse = ", "), "
")
  cat(sprintf("n = %d (train %d, test %d); responders: train %.0f%%, test %.0f%%
", nrow(d), nrow(tr), nrow(te),
              100 * mean(tr$.resp), 100 * mean(te$.resp)))
  cat("Phenotype mix (%), train vs test:", paste(sprintf("%s %.0f/%.0f", levels(d$phenotype), 100 * prop.table
(table(tr$phenotype)), 100 * prop.table(table(te$phenotype))), collapse = "; "), "
")

  # Understand (training data only). EDA: distribution of every numeric covariate, a count table for
```

```r
  # every categorical one, and the correlation matrix, each per arm.
  eda_num <- bind_rows(lapply(num_a, function(v) { x <- tr[[v]]
    data.frame(covariate = v, n = length(x), mean = mean(x), sd = sd(x), median = median(x), min = min(x), max = max(x),
               pct_at_min = 100 * mean(x == min(x))) }))
  write.csv(mutate(eda_num, across(where(is.numeric), ~ round(.x, 2))), sprintf("%s/eda_numeric_%s.csv", OUT, A$name), row.names = FALSE)
  eda_cat <- bind_rows(lapply(intersect(cat_cov, covs_a), function(v)
    data.frame(covariate = v, level = names(table(tr[[v]])), n = as.integer(table(tr[[v]])), pct = round(100 * as.numeric(prop.table(table(tr[[v]]))))))
  write.csv(eda_cat, sprintf("%s/eda_categorical_%s.csv", OUT, A$name), row.names = FALSE)
  cat("
Categorical covariates (training patients):
"); print(eda_cat, row.names = FALSE)
  ggsave(sprintf("%s/fig_eda_distributions_%s.png", OUT, A$name),
    ggplot(pivot_longer(tr[num_a], everything()), aes(value)) + geom_histogram(bins = 25, fill = "#2c7fb8", colour = "white") +
      facet_wrap(~name, scales = "free", ncol = 4) + labs(x = NULL, y = "Patients", title = paste(A$name, ": numeric covariates, training patients")),
    width = 11, height = 8, dpi = 150)
  cm_full <- cor(tr[num_a], method = "spearman")
  ggsave(sprintf("%s/fig_eda_correlation_%s.png", OUT, A$name),
    ggplot(as.data.frame(as.table(cm_full)), aes(Var1, Var2, fill = Freq)) + geom_tile() +
      geom_text(aes(label = sprintf("%.2f", Freq)), size = 2.6) +
      scale_fill_gradient2(low = "#b2182b", mid = "white", high = "#2166ac", limits = c(-1, 1), name = "Spearman") +
      labs(x = NULL, y = NULL, title = paste(A$name, ": correlation between numeric covariates, training patients")) +
      theme(axis.text.x = element_text(angle = 40, hjust = 1)), width = 9, height = 8, dpi = 150)
  cm <- cor(tr[num_a]); cm[upper.tri(cm, diag = TRUE)] <- NA
  hi <- which(abs(cm) > 0.7, arr.ind = TRUE)
  if (nrow(hi)) cat("Correlated (|r| > 0.7):", paste(sprintf("%s ~ %s (%.2f)", rownames(cm)[hi[, 1]], colnames(cm)[hi[, 2]], cm[hi]), collapse = "; "), "
")

  # Is a categorical covariate just a binned copy of a numeric one? Share of each numeric covariate's
  # variance explained by each categorical one (only strong links are printed).
  for (cv in intersect(cat_cov, covs_a)) for (nv in num_a) { r2 <- summary(lm(tr[[nv]] ~ tr[[cv]]))$r.squared
    if (r2 > 0.3) cat(sprintf("Linked: %s explains %.0f%% of the variation in %s
", cv, 100 * r2, nv)) }

  # Negative control: a pure-noise covariate. Anything ranked at or below it is treated as noise.
  set.seed(SEED); tr$noise_probe <- rnorm(nrow(tr)); te$noise_probe <- rnorm(nrow(te))
  vars <- c(covs_a, "noise_probe")

  reg <- fit_all(tr, te, vars, A$outcome, "gaussian")
  cls <- fit_all(tr, te, vars, ".resp", "binomial")

  # Stability: how often LASSO keeps each covariate across 200 bootstrap resamples of the training set.
  Xtr <- design(tr, vars); set.seed(SEED)
  stab <- rowMeans(replicate(200, { i <- sample(nrow(tr), replace = TRUE)
    cv <- cv.glmnet(Xtr[i, ], tr[[A$outcome]][i], alpha = 1, nfolds = 5)
    by_var(as.numeric(coef(cv, s = "lambda.1se"))[-1], Xtr)[vars] > 0 }))
  # One vote per method. The two ML votes are "ranked above the noise probe" (kickoff slides 72 and 74).
  s <- reg$sel
  tab <- data.frame(covariate = vars,
    Stepwise = s$Stepwise > 0, LASSO = s$LASSO > 0, `Elastic net` = s$`Elastic net` > 0,
    `Random forest` = s$`Random forest` > s["noise_probe", "Random forest"],
    XGBoost = s$`XGBoost SHAP` > s["noise_probe", "XGBoost SHAP"],
    lasso_stability_pct = round(100 * stab), check.names = FALSE)
  tab$votes <- rowSums(tab[2:6])
  # "signal" needs near-unanimity and stability; "uncertain" needs two votes and more stability than the
  # probe shows (about 10%); the rest is noise. The two ML votes alone are a low bar, hence the stability test.
  tab$call  <- ifelse(tab$votes >= 4 & tab$lasso_stability_pct >= 60, "signal",
               ifelse(tab$votes >= 2 & tab$lasso_stability_pct >= 20, "uncertain", "noise"))
  tab$call[tab$covariate == "noise_probe"] <- "negative control"
  tab <- tab[order(-tab$votes, -tab$lasso_stability_pct), ]

  # Direction and size: linear model on the consensus covariates, numeric ones per 1 SD.
  keep <- setdiff(tab$covariate[tab$call == "signal"], "noise_probe")
  trs <- tr; tes <- te
```

```
  for (v in num_a) { m <- mean(tr[[v]]); sdv <- sd(tr[[v]]); trs[[v]] <- (tr[[v]] - m) / sdv; tes[[v]] <- (te[[v]] - m) / sdv }
  if (length(keep)) {
    cons <- lm(reformulate(keep, A$outcome), trs)
    eff  <- as.data.frame(cbind(coef(cons), confint(cons)))[-1, ]; names(eff) <- c("estimate", "lo", "hi"); eff$term <- rownames(eff)
    reg$pred[["Consensus linear"]] <- predict(cons, tes)
    cls$pred[["Consensus linear"]] <- predict(glm(reformulate(keep, ".resp"), binomial, trs), tes, type = "response")
    write.csv(eff, sprintf("%s/effects_%s.csv", OUT, A$name), row.names = FALSE)
    # Same covariates, but the top one enters as above/below a cut-off. The cut-off is the first split of a
    # one-level regression tree on training patients, so the test set plays no part in choosing it.
    top <- keep[1]
    if (top %in% num_a) {
      stump <- rpart::rpart(reformulate(top, A$outcome), tr, control = rpart::rpart.control(maxdepth = 1, minbucket = 15, cp = 0))
      cut_at <- stump$splits[1, "index"]
      trc <- trs; tec <- tes; trc$.below <- tr[[top]] < cut_at; tec$.below <- te[[top]] < cut_at
      rhs <- c(".below", keep[-1])
      reg$pred[["Linear with cut-off"]] <- predict(lm(reformulate(rhs, A$outcome), trc), tec)
      cls$pred[["Linear with cut-off"]] <- predict(glm(reformulate(rhs, ".resp"), binomial, trc), tec, type = "response")
      cat(sprintf("Cut-off model: %s below %.2f (chosen on training patients)
", top, cut_at))
    }
    ggsave(sprintf("%s/fig_effects_%s.png", OUT, A$name),
      ggplot(eff, aes(estimate, reorder(term, estimate))) + geom_vline(xintercept = 0, linetype = 2) +
        geom_pointrange(aes(xmin = lo, xmax = hi)) +
        labs(x = sprintf("Change in %s per 1 SD; %s = better", A$outcome, A$better), y = NULL,
             title = paste(A$name, ": consensus predictors of response")), width = 7, height = 3.5, dpi = 150)
  } else cat("No covariate reached consensus in this arm.
")
  tab$arm <- A$name; all_cov[[a]] <- tab
  write.csv(tab, sprintf("%s/covariates_%s.csv", OUT, A$name), row.names = FALSE)
  cat("
Covariate calls:
"); print(tab[c("covariate", "votes", "lasso_stability_pct", "call")], row.names = FALSE)

  # Evaluate once on the locked test set.
  yt <- te[[A$outcome]]; rt <- te$.resp
  cut_p <- mean(tr$.resp)     # responder cut-off: the training responder share, fixed before seeing the test set
  board <- bind_rows(lapply(names(reg$pred), function(m) {
    r <- reg_metrics(yt, reg$pred[[m]]); k <- cls_metrics(rt, cls$pred[[m]], cut = cut_p)
    data.frame(arm = A$name, method = m, type = type_of[[m]], RMSE = r[["RMSE"]], MAE = r[["MAE"]], R2 = r[["R2"]],
               R2_95CI = boot_ci(yt, reg$pred[[m]], function(y, p) reg_metrics(y, p)[["R2"]]),
               AUC = k[["AUC"]], AUC_95CI = boot_ci(rt, cls$pred[[m]], auc),
               Recall = k[["Recall"]], Precision = k[["Precision"]], Accuracy = k[["Accuracy"]]) }))
  board <- board[order(board$type, -board$R2), ]
  all_board[[a]] <- board
  write.csv(board, sprintf("%s/scoreboard_%s.csv", OUT, A$name), row.names = FALSE)
  cat(sprintf("
Scoreboard (test n = %d, %d responders; responder cut-off %.2f; XGBoost tuned: %s reg / %s cls):
",
              nrow(te), sum(rt), cut_p, reg$tuned, cls$tuned))
  print(mutate(board, across(where(is.numeric), ~ round(.x, 2))), row.names = FALSE)

  # SHAP summary for the tuned XGBoost model (training data): size and direction per covariate.
  ggsave(sprintf("%s/fig_shap_%s.png", OUT, A$name),
    shapviz::sv_importance(shapviz::shapviz(reg$xgb, X_pred = reg$Xtr), kind = "beeswarm", max_display = 10) +
      labs(title = paste(A$name, ": XGBoost SHAP values for", A$outcome)), width = 7, height = 4.5, dpi = 150)

  # A two-level decision tree as a read-aloud stratification rule, checked on held-out patients.
  # Built only from the consensus covariates, so the rule cannot split on noise.
  tree <- rpart::rpart(reformulate(if (length(keep)) keep else covs_a, ".resp"), data = tr, method = "class",
                       control = rpart::rpart.control(maxdepth = 2, minbucket = 15, cp = 0.02))
  png(sprintf("%s/fig_tree_%s.png", OUT, A$name), width = 1000, height = 650, res = 150)
  rpart.plot::rpart.plot(tree, type = 4, extra = 104, main = paste(A$name, ": responder rule (training data)"))
  dev.off()
```

```
  leaf <- function(d) rownames(tree$frame)[rpart:::pred.rpart(tree, rpart:::rpart.matrix(model.frame(delete.response(terms(tree)), d, na.action = na.pass)))]
  tl <- data.frame(leaf = leaf(te), resp = rt) |> group_by(leaf) |> summarise(test_n = n(), test_responder_pct = round(100 * mean(resp)), .groups = "drop")
  tl$rule <- sapply(tl$leaf, function(l) paste(rpart::path.rpart(tree, as.numeric(l), print.it = FALSE)[[1]][-1], collapse = " & "))
  write.csv(tl, sprintf("%s/tree_%s.csv", OUT, A$name), row.names = FALSE)
  cat("
Tree rule, responder rate in held-out patients:
"); print(as.data.frame(tl[c("rule", "test_n", "test_responder_pct")]), row.names = FALSE)

  # Stratification: do held-out patients ranked by predicted benefit actually differ?
  best <- if (length(keep)) "Consensus linear" else "LASSO"
  p <- reg$pred[[best]]; if (A$better == "lower") p <- -p
  st <- data.frame(group = cut(rank(p, ties.method = "first"), 3, labels = c("Predicted low", "Predicted middle", "Predicted high")),
                   y = yt, resp = rt) |> group_by(group) |>
    summarise(n = n(), mean_outcome = mean(y), responder_pct = 100 * mean(resp), .groups = "drop")
  write.csv(st, sprintf("%s/strata_%s.csv", OUT, A$name), row.names = FALSE)
  ggsave(sprintf("%s/fig_strata_%s.png", OUT, A$name),
    ggplot(st, aes(group, responder_pct)) + geom_col(fill = "#2c7fb8") +
      geom_text(aes(label = sprintf("%.0f%%
(n=%d)", responder_pct, n)), vjust = -0.2) + ylim(0, 115) +
      labs(x = paste("Held-out patients grouped by predicted benefit,", best, "model"), y = "Observed responders (%)",
           title = paste(A$name, ": predicted benefit vs observed response")), width = 7, height = 4, dpi = 150)
}

# ---- Cross-arm summaries ----
cov_all <- bind_rows(all_cov); board_all <- bind_rows(all_board)
write.csv(cov_all, file.path(OUT, "covariates_all_arms.csv"), row.names = FALSE)
write.csv(board_all, file.path(OUT, "scoreboard_all_arms.csv"), row.names = FALSE)

dm <- cov_all |> pivot_longer(c(Stepwise, LASSO, `Elastic net`, `Random forest`, XGBoost), names_to = "method", values_to = "picked") |>
  mutate(method = factor(method, c("Stepwise", "LASSO", "Elastic net", "Random forest", "XGBoost")), covariate = reorder(covariate, votes, sum))
ggsave(file.path(OUT, "fig_covariate_matrix.png"),
  ggplot(dm, aes(method, covariate)) + geom_point(aes(shape = picked, colour = picked), size = 3) +
    scale_shape_manual(values = c(`TRUE` = 16, `FALSE` = 1)) + scale_colour_manual(values = c(`TRUE` = "#08519c", `FALSE` = "grey75")) +
    facet_wrap(~arm) + guides(shape = "none", colour = "none") +
    labs(x = NULL, y = NULL, title = "Covariates picked by each method (filled = picked)",
         subtitle = "Random forest = permutation importance above the noise probe; XGBoost = SHAP above the noise probe") +
    theme(axis.text.x = element_text(angle = 35, hjust = 1)), width = 11, height = 6, dpi = 150)
ggsave(file.path(OUT, "fig_scoreboard.png"),
  ggplot(board_all, aes(R2, reorder(method, R2), fill = type)) + geom_col() + facet_wrap(~arm) + geom_vline(xintercept = 0) +
    scale_fill_manual(values = c(Covariate = "#2c7fb8", ML = "#d95f0e")) +
    labs(x = "Test-set R² (continuous outcome)", y = NULL, fill = NULL, title = "Covariate models vs machine learning, held-out patients"),
  width = 11, height = 4, dpi = 150)

cat("
Signal covariates per arm:
")
print(as.data.frame(cov_all |> filter(call == "signal") |> group_by(arm) |> summarise(signal = paste(covariate, collapse = ", "))), row.names = FALSE)
writeLines(capture.output(sessionInfo()), file.path(OUT, "session_info.txt"))   # versions, for reproducibility
cat("
Done. Outputs in", OUT, "
")

```